THE UNIVERSITY OF WINNIPEG

# APPLIED COMPUTER SCIENCE

Course Number - ACS-3931-001 Course Name – Principles of Operating Systems

**<u>Assignment #3</u> 'c' programming for process output redirection, 'c' programming the bounded-buffer producer / consumer algorithm, and book problems.**
**Today's Date:** March 3th, 2026 midnight (Nexus clock)
**Due Date:** March 31st, 2026 – 11:59:00 pm (Nexus clock)

**IMPORTANT NOTES:**

**Submit all assignment results as described below in one Zip file via Nexus named 'assignment3_<studentID>.zip'.**

**Create only one pdf for all screenshots and text report(s) identifying clearly the Part number the screenshot / text that they belong to. If more than one pdf file is in the zip file, then 25 points will be deducted.**

**We are checking that mars and/or Pi screenshots have your username (in mars login) or student id number (in pi hostname) respectively to uniquely identify your ssh session when you run your programs, otherwise it will not be accepted as deliverable.**

**The assignment is worth 100 points + 20% optional if assignment is done on Raspberry Pi, e.g. if one gets 75 then 20% will be added to mark 75 or 15 pts in this case for a final mark of 90 pts total. One can get more than 100 pts for an assignment**

**Part I (50 points)**
In Part I, you will practice implementing process redirection by implementing a small c program that redirects standard out to a file. We are going to use dup2() system call to redirect the standard out to a file.

The c program of Part I must do the following:

a) You will write a c program named 'assign03_red.c'. This program must

- Opens a file, whose filename was passed as argument. The program either creates the file (if it does not already exist) or truncates its length to 0 (if it does exist)

- If the file is created, the access permission bits in the file mode of the file are set to permit reading and writing by the owner, and to permit reading only by group members and others

- The filename will be specified as a command line argument. If no filename is passed as argument, and error condition is detected, a print message 'Error: no filename passed!' is output and the program exits returning -1 as the error code

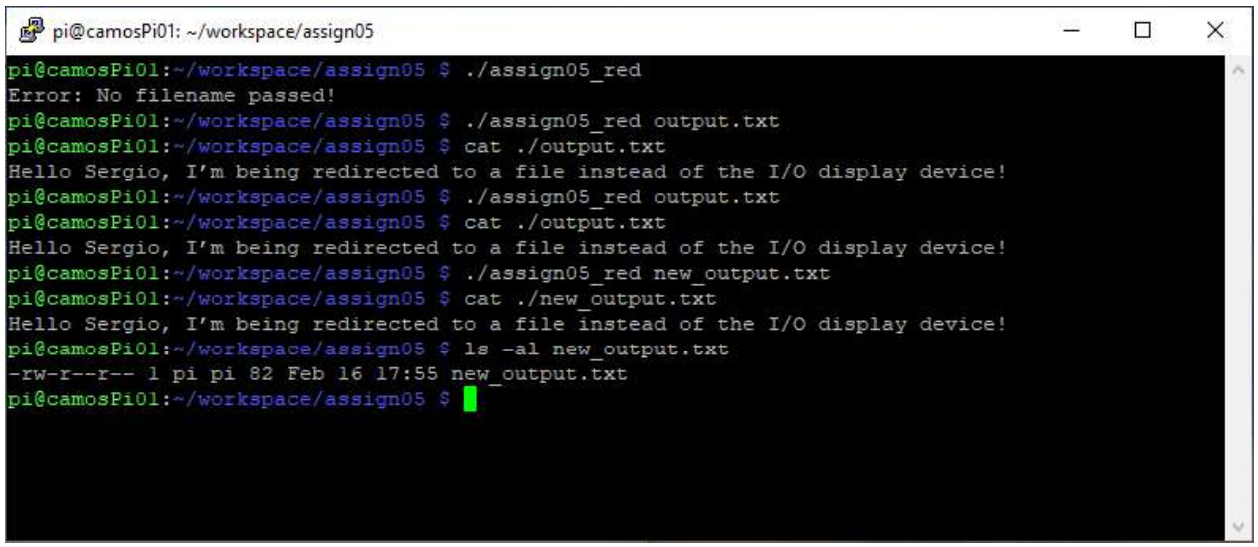
- The program writes the output message *'Hello <my name> I'm being redirected to a file instead of the I/O display device!'* to the file using standard out

- Remember, the output must be sent to standard out.

- Compile, link and run your 'assign03_red.c' program

- Test your program and analyze the results for the following scenarios
    - No filename passed
    - Filename passed of an existing file and then displaying file contents with 'cat'
    - Filename passed of a new file and then displaying file contents with 'cat'
    - List new file permissions with 'ls -al <new_filename>'

Below a screenshot sample of a typical test run. NOTE: use assign03_red instead of assign05_red




Submit via Nexus the following for Part I of the assignment:

- Source code for c program named 'assign03_red.c'. Important to add your name and student number as a comment at the beginning of the *.c file
- Briefly explain in a small paragraph (2 or 3 lines) what dup2() system call help you to achieve in your c program. Put the paragraph in a pdf named assign03PartI.pdf
- Briefly explain in a small paragraph (2 or 3 lines) what are the benefits of file descriptor duplication obtained with dup2 or dup system calls? Put the paragraph in a pdf named assign03PartI.pdf
- Screenshot of your program running the test scenarios as shown in the previous screenshot and add it to the assign03PartI.pdf
- Put all these files (*.c program, pdf file) as Part I in the assignment3.zip file

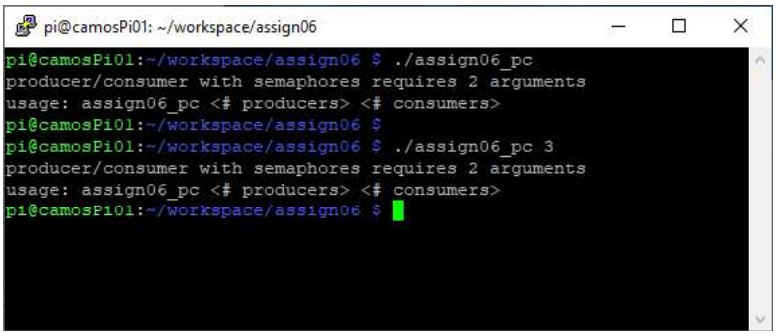
**Part II (50 points)**

In this part you will get some experience in implementing commonly use OS algorithms. Thus, you will implement in c language the algorithm for the bounded-buffer producer / consumer problem using semaphores, which was discussed in class. This algorithm is in the textbook Figure 5.16, and in Topic IIc pdf document Slide 48. Also, in class we have discussed a simple program boundedBufferProducerConsumerProblemwSem.c that you can take as a basis.

The c program of Part I must do the following:

a) You will write a c program named 'assign03_pc.c'. This program must

- Follow the algorithm logic of Topic IIc Slide 48 discussed in class.
- The program dynamically creates the number of producers and consumers that exist in the program execution. These values are passed as arguments when executing the program. The first argument is the number of producers and the second argument is the number of consumers. If less arguments are passed, the program should indicate error message with usage pattern and quit. This is shown next. NOTE: use assign03_pc instead of assign06_pc




- Every producer and consumer should be in separate threads. For example, if you have 3 producers and 2 consumers, then you will create dynamically 5 threads, one for each.
    - The thread method for the producer is defined as

      void* thread_producer(void* arg)

    - The thread method for the consumer is defined as

      void* thread_consumer(void* arg)

- There are several constants defined as a minimum:
    - #define BUFFER_SIZE 10 <- this determines the number of buffer slots
    - #define BUFFER_ITEM_SIZE 10 <- this determines the buffer slot size in # chars
    - #define SLEEP_TIME_SECS 2 <- this determines secs to wait before a producer generates a new data for the buffer
    - #define DATA_NUMBER_TO_PRODUCE 5 <- this determines how many data units a producer will generate before dying

- Every time a producer generates data, a message should be printed <u>outside the critical section</u> that clearly identifies the producer and the data item produced. If a producer generates 5 items (as per constant DATA_NUMBER_TO_PRODUCE), then the data items produced will go from 0 to 4. Similarly, the producer number will go from 0 to the number of producers passed in the program argument run. The producer message follows the syntax:

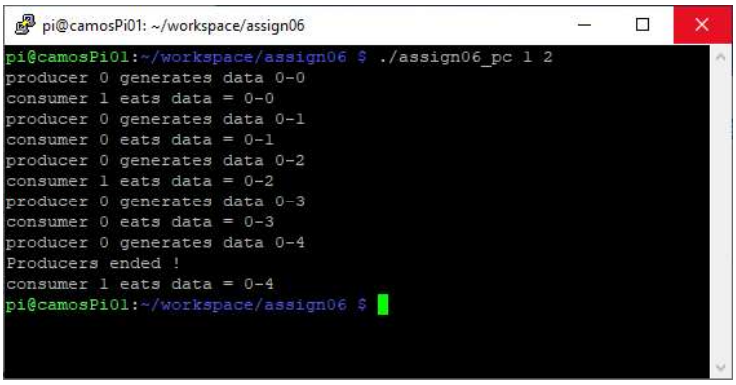
  $ producer <producer_#> generates data <producer_#>-<data_item_#>

  This producer message is shown in the next screenshot for '1 producer, 2 consumers' run

- Every time a consumer eats data, a message should be printed <u>outside the critical section</u> that clearly identifies the consumer and the data item that it is being eaten from the buffer. The consumer number will go from 0 to the number of consumers passed in the program argument run. The consumer message follows the syntax:

  $ consumer <consumer_#> eats data = <producer_#>-<data_item_#>

  This consumer message is also shown in the next screenshot for a '1 producer, 2 consumers' run




- The main method should wait until the consumer threads eat all data items that were generated by all producer threads. Once all data is eaten, the main method should quit.
- When all producer threads complete, the main method should print the message "Producers ended!"
- All used semaphores should be destroyed before the main method ends.
- A screenshot run is shown next with 3 producers and 1 consumer. Note that 15 data items were produced, and all were eaten by the lonely consumer.

```
pi@camosPi01: ~/workspace/assign06
pi@camosPi01:~/workspace/assign06 $ ./assign06_pc 3 1
producer 2 generates data 2-0
producer 1 generates data 1-0
producer 0 generates data 0-0
consumer 0 eats data = 2-0
consumer 0 eats data = 1-0
consumer 0 eats data = 0-0
producer 2 generates data 2-1
producer 1 generates data 1-1
consumer 0 eats data = 2-1
consumer 0 eats data = 1-1
consumer 0 eats data = 0-1
producer 0 generates data 0-1
producer 2 generates data 2-2
consumer 0 eats data = 2-2
consumer 0 eats data = 1-2
producer 1 generates data 1-2
producer 0 generates data 0-2
consumer 0 eats data = 0-2
producer 2 generates data 2-3
consumer 0 eats data = 2-3
producer 1 generates data 1-3
consumer 0 eats data = 1-3
consumer 0 eats data = 0-3
producer 0 generates data 0-3
producer 2 generates data 2-4
consumer 0 eats data = 2-4
producer 1 generates data 1-4
producer 0 generates data 0-4
Producers ended !
consumer 0 eats data = 1-4
consumer 0 eats data = 0-4
pi@camosPi01:~/workspace/assign06 $
```

- Another screenshot run with 3 producers and 4 consumers is shown next

```
pi@camosPi01: ~/workspace/assign06
pi@camosPi01:~/workspace/assign06 $ ./assign06_pc 3 4
producer 2 generates data 2-0
producer 1 generates data 1-0
producer 0 generates data 0-0
consumer 0 eats data = 2-0
consumer 1 eats data = 1-0
consumer 2 eats data = 0-0
producer 2 generates data 2-1
producer 1 generates data 1-1
consumer 3 eats data = 2-1
producer 0 generates data 0-1
consumer 0 eats data = 1-1
consumer 1 eats data = 0-1
producer 2 generates data 2-2
consumer 2 eats data = 2-2
producer 1 generates data 1-2
consumer 3 eats data = 1-2
producer 0 generates data 0-2
consumer 0 eats data = 0-2
producer 2 generates data 2-3
consumer 1 eats data = 2-3
producer 1 generates data 1-3
consumer 2 eats data = 1-3
producer 0 generates data 0-3
consumer 3 eats data = 0-3
producer 2 generates data 2-4
consumer 0 eats data = 2-4
producer 1 generates data 1-4
consumer 1 eats data = 1-4
producer 0 generates data 0-4
Producers ended !
consumer 2 eats data = 0-4
pi@camosPi01:~/workspace/assign06 $
```

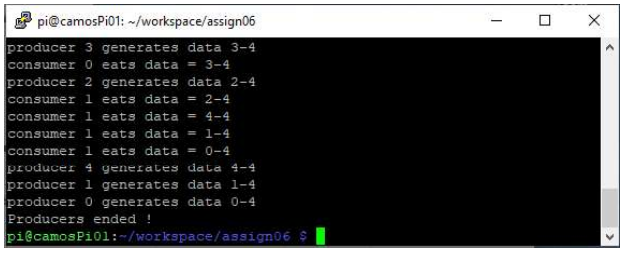
- Compile, link and run your ‘assign03_pc.c’ program

- Test your program and analyze the results for the following scenarios
    - No arguments passed
    - One argument passed
    - Run your program with the following producer / consumer combinations

| # Producers | # Consumers |
|---|---|
| 1 | 2 |
| 3 | 1 |
| 3 | 4 |
| 0 | 3 |
| 4 | 0 |
| 5 | 5 |

Submit via Nexus the following for Part II of the assignment:

1. Source code for c program named ‘assign03_pc.c’. Important to add your name and student number as a comment at the beginning of the *.c file
2. Briefly explain in a small paragraph (2 or 3 lines) why sometimes you get a producer message of a data item what was already eaten by a consumer as shown in the next piece of screenshot. For example, note data items 4-4, 1-4, and 0-4 in the screenshot below. Put the paragraph labeled “Question 2” in a pdf named assign03PartII.pdf




3. Briefly explain (2 or 3 lines) what happens in the combination “0 producers, 3 consumers” in your program. Put the paragraph labeled “Question 3” in a pdf named assign03PartII.pdf
4. Briefly explain (2 or 3 lines) what happens in the combination “4 producers, 0 consumers” in your program. Put the paragraph labeled “Question 4” in a pdf named assign03PartII.pdf
5. Screenshot of your program running the test scenario combinations given in the previous table as shown in the preceding screenshots and add it to the assign03PartII.pdf. Labeled the screenshots as “Question 5” in the pdf.
6. Put all these files (*.c program, pdf file) as Part II in the assignment3 zip file

**Part III (0 points) - Nothing to report for Part III. This is additional material for final exam study.**

You will do some problems from your Stallings OS textbook 9th US edition (No global edition).

a) Read textbook as follows.

- Chapter 5 all sections
- Chapter 6 sections 6.1 to 6.8

b) Work on the following book problems for exam practice.

1. Problem 5.6, only part a) Page 250
2. Problem 5.8 Page 251
3. Problem 5.24 a) b) c) and d) Page 258
4. Problem 6.5 only b) and c), page 305
5. Problem 6.14 Page 307
6. Problem 5.9 Page 251
7. Problem 5.15 Page 253
8. Problem 6.3 only provide narrative for paths 3 and 5, page 304
9. Problem 6.6 only a) Page 305